

✨ Member-only story

Master Agentic Engineering: The 3-Pillar System That Changes How We Build Software

Learn the 3-pillar orchestration system that transforms AI-assisted coding. Specialized agents, unified control, and observability patterns that change how we build software — from context chaos to coordinated engineering.

12 min read · Nov 5, 2025



Reza Rezvani

Following ▾



Listen



Share



More

THE NEW ENGINEERING



Agentic Engineering
Multi-Agent Orchestration

Agentic Engineering — The 3 Pillars of the orchestration system

When Seven Browser Tabs Broke My Development Flow

Last week. I'm deep in a refactoring task when my flow state shatters.

I have Claude Code open in one tab, explaining architecture patterns. Copilot in VSCode, suggesting completions. ChatGPT in another tab, generating documentation. Cursor in yet another window, helping with a complex regex. Each tool brilliant in isolation. Each one completely unaware of what the others are doing.

I spend the next twenty minutes re-explaining the same database schema to three different AI tools. By the time I've synchronized their contexts, I've forgotten what I was actually building.

This is the hidden problem with AI-assisted development: *We've moved from one bottleneck to another.* The AI can write code faster than we can think. But coordinating multiple AI tools? That's where we're bleeding velocity.

Then I discovered something that changed everything: A small group of engineering teams had stopped fighting this problem. They weren't collecting more agents. They were **orchestrating** them.

Let me show you the three-pillar system they're using — and how it fundamentally changes the way you build software.

Master Claude Agent SDK: A 5-Step Integration Guide to Cut Development Time 70% with...

Stop building custom agent loops from scratch. Follow this battle-tested integration to deploy secure, scalable AI...

alirezarezvani.medium.com



. . .

The Invisible Velocity Killer: Context Pollution

Single-agent systems hit a hard ceiling around five hundred lines of code. Beyond that, the context window gets polluted. The agent starts confusing React patterns with Express middleware. It hallucinates dependencies. It suggests solutions that worked three files ago but don't apply here.

Managing AI context becomes some times harder than writing code manually.

But here's what most of us miss: *This problem was already solved in distributed systems design.* We just haven't applied those patterns to agents yet.

. . .

The Orchestration Paradigm: Specialized Agents, Unified Control

Remember when everyone was building monolithic applications? Then microservices revolutionized everything. Not because microservices were inherently better, but because they **matched how complex systems actually work**.

Orchestrated agents are the same shift.

Instead of one general-purpose agent that tries to do everything, you compose specialized agents that excel at specific tasks. A security auditor that only reviews code. A testing agent that only writes tests. A documentation agent that only updates docs.

The breakthrough is in the coordination layer.

Here's the architecture:



Each agent maintains its own isolated context. The security agent never gets confused by frontend code. The testing agent doesn't hallucinate backend logic. The orchestrator handles the coordination — just like Kubernetes coordinates containers.

Let me show you what this looks like in production.

. . .

Pillar 1: The Orchestrator Agent — Your Development Control Plane

The orchestrator agent is your single interface to the entire agent fleet. Think of it as the conductor of an orchestra — each musician is brilliant, but the conductor makes them work together.

Here's how I implement this with Claude Code:

```
# .claude/agents/security-auditor.md
---
name: "Security Auditor"
description: "Reviews code for vulnerabilities"
tools: ["view"] # Read-only
system_prompt: |
  You scan code for security issues:
  - Hardcoded secrets or API keys
  - SQL injection risks (non-parameterized queries)
  - XSS vulnerabilities (unsanitized input)
  - Weak authentication patterns

  Output: File:line references with severity levels
context_isolation: true
---
```

Notice what this agent **cannot** do: It has no write access. It can't modify code. It can't run commands. It's a pure audit tool. This is deliberate.

Now compare it to the implementation agent:

```
# .claude/agents/backend-implementer.md
---
name: "Backend Implementer"
description: "Implements Node.js/Express features"
tools: ["view", "str_replace", "create_file"]
system_prompt: |
  You implement backend features in Express/TypeScript.
  - Follow patterns in src/middleware/
  - Use AsyncHandler for error wrapping
  - Write integration tests alongside code
  - Create branch, never push to main
context_isolation: true
max_parallel: 2
---
```

This agent can write code. But it still can't push to main. It creates a branch and leaves the merge decision to humans.

Here's what happens when I run a complex task:

```
claude code "Add JWT authentication to the API"
```

The orchestrator:

1. Analyzes the codebase structure
2. Identifies affected areas (middleware, routes, tests, docs)
3. Delegates to specialized agents **in parallel**
4. Validates that changes don't conflict
5. Presents a consolidated pull request

I recently used this for a multi-file authentication refactor. The orchestrator coordinated changes across fourteen files, three languages, and multiple architectural layers. Perfect context preservation. Zero confusion between frontend and backend patterns.

The difference in developer experience is profound. I'm not babysitting AI tools anymore. I'm **delegating outcomes, not managing tasks.**

Agentic Thinking — The Orchestration is the New Engineering

• • •

Pillar 2: CRUD for Agents — Version Control for Your Workforce

Once you have specialized agents, you need to manage them like code. Because that's what they are — executable specifications.

Here's my agent directory structure:

```
.claude/
├── agents/
│   ├── security-auditor.md
│   ├── backend-implementer.md
│   ├── frontend-specialist.md
│   ├── api-doc-writer.md
│   ├── test-driven-dev.md
│   └── performance-profiler.md
└── commands/
```

```
|   |— new-api-endpoint.md
|   |— react-component-suite.md
|   |— full-feature-flow.md
|— .mcp.json
```

Create: New specialized agent in minutes

```
cat > .claude/agents/db-migration-specialist.md << 'EOF'
---
name: "Database Migration Specialist"
description: "Creates Postgres migrations"
tools: ["bash", "create_file", "view"]
system_prompt: |
  Create Postgres migrations using node-pg-migrate.
  - Always include both up() and down()
  - Test with: npm run migrate:test
  - Never modify existing migrations
  - Naming: YYYYMMDDHHMMSS_description.js
---
EOF
```

Read: Query agent capabilities

```
# List available agents
ls -la .claude/agents/

# Check specific config
cat .claude/agents/security-auditor.md
```

Update: Refine agent behavior

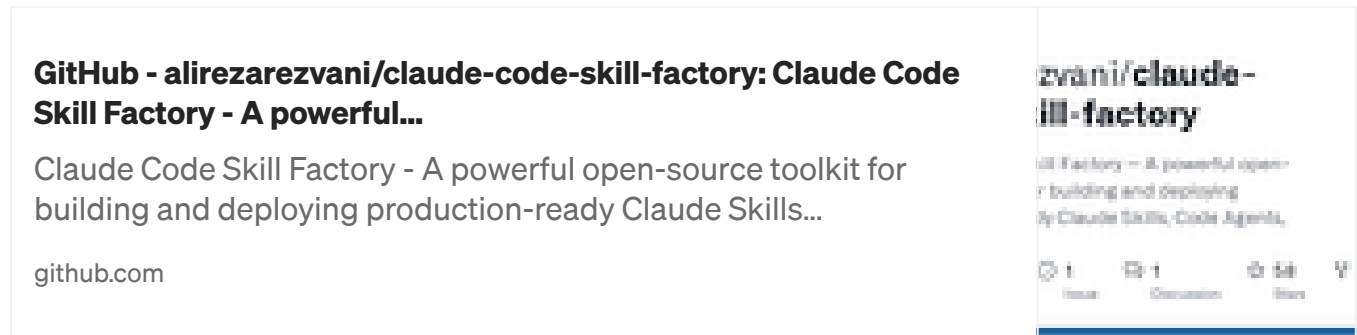
When an agent isn't performing well, I edit its system prompt. Last week, my security agent kept flagging false positives on GraphQL queries. I updated the prompt to understand our specific GraphQL setup. Problem solved.

Delete: Retire obsolete agents

When we switched testing frameworks, I removed the old testing agent and created a new one for our current stack.

The key insight here: Agents become team assets. Not personal configurations. Anyone on the team can use the security auditor. Anyone can leverage the performance profiler. We version them in git. We share them via `.mcp.json`. We review changes to agent configs like we review code.

This is **infrastructure as code** applied to AI agents.



. . .

The Permission Model: Security By Design

Let me show you why explicit permissions matter.

Last month, an agent tried to execute `git push --force` on our main branch. Here's how we caught it:

```
# Orchestrator security configuration
dangerous_commands_blocklist:
  - "rm -rf"
  - "git push --force"
  - "sudo"
  - "DROP TABLE"

sensitive_actions_require_confirmation:
  - "git push"
  - "npm publish"
  - "terraform apply"
```

The orchestrator intercepted the force push and prompted: *“Agent attempting force push to protected branch. Approve? [y/N]”*

I hit “N” and investigated. The agent had misunderstood a merge conflict and thought force-pushing was the solution. This is why **we start from deny-all and allowlist explicitly**.

Here’s how I think about agent permissions:

Read-only agents (*security, documentation, analysis*):

```
tools: ["view"]
```

Write-limited agents (implementation, refactoring):

```
tools: ["view", "str_replace"]
```

Full-access agents (testing, deployment — with confirmation):

```
tools: ["bash", "view", "str_replace", "create_file"]  
sensitive_actions_require_confirmation: true
```

This isn’t paranoia. It’s engineering discipline. You wouldn’t give a junior developer production database access on day one. Don’t give agents unrestricted access either.

. . .

Pillar 3: Observability — Understanding Your Agent Fleet

Most teams run agents blind. They don’t know which agents are efficient. Which are costing too much. Which are failing silently.

I built a simple observability layer that logs every agent action:

```
// agent-logger.js
const fs = require('fs');

function logAgentAction(data) {
  const entry = {
    timestamp: new Date().toISOString(),
    agent: data.agentName,
    action: data.actionType,
    input_tokens: data.inputTokens,
    output_tokens: data.outputTokens,
    success: data.success,
    duration_ms: data.duration,
    files_touched: data.filesModified,
    tools_used: data.toolCalls.map(t => t.name)
  };

  fs.appendFileSync(
    '.claude/agent-actions.log',
    JSON.stringify(entry) + '\n'
  );
}
```

Then I analyze the logs weekly:

```
# analyze-agents.py
import json
from collections import defaultdict

actions = defaultdict(list)
with open('.claude/agent-actions.log') as f:
    for line in f:
        action = json.loads(line)
        actions[action['agent']].append(action)
for agent, logs in actions.items():
    success_rate = sum(l['success'] for l in logs) / len(logs)
    avg_tokens = sum(l['input_tokens'] + l['output_tokens']
                     for l in logs) / len(logs)

    print(f"{agent}:")
    print(f"  Tasks: {len(logs)}")
    print(f"  Success rate: {success_rate:.1%}")
    print(f"  Avg tokens: {avg_tokens:,.0f}\n")
```

Real debugging scenario:

My frontend specialist agent started failing repeatedly. The logs revealed the pattern:

Failed tasks:

- "Add dark mode toggle" → Token limit exceeded
- "Refactor styling" → Token limit exceeded
- "Update color palette" → Token limit exceeded

All styling-related. All hitting token limits.

The fix was architectural: I split one bloated agent into two specialized agents.

```
# Before: One agent trying to do everything
frontend-specialist.md (11K tokens average)
```

```
# After: Two focused agents
react-component-builder.md (3.2K tokens)
styling-specialist.md (2.8K tokens)
```

The component builder handles structure. The styling specialist handles CSS/Tailwind. They coordinate through the orchestrator. Failure rate dropped from twenty-six percent to three percent.

This is what observability enables: Data-driven agent optimization. Not guessing. Not hoping. Measuring and improving systematically.

Git Worktrees + Claude Code: Parallel AI Development Guide

Stop context switching. Learn how engineering teams use git worktrees with Claude Code for parallel development...

alirezarezvani.medium.com



The New Paradigm: How Software Development Actually Changes

Let me be direct about what this isn't: This isn't about replacing developers. It's not about autonomous AI that writes entire applications while you sleep.

It's about changing your role from implementer to architect.

With orchestrated agents, I spend my time:

- Designing system architecture
- Making algorithmic decisions
- Reviewing agent-generated code
- Refining agent specifications
- Optimizing the orchestration patterns

I don't spend time:

- Manually refactoring across multiple files
- Writing boilerplate integration tests
- Updating documentation in eight places
- Implementing CRUD endpoints for the hundredth time

The quality of code improves because specialized agents are consistent. My testing agent always writes integration tests. Always. My security agent never misses hardcoded credentials. Never. My documentation agent always updates the OpenAPI spec when routes change.

Humans are inconsistent. We forget. We get tired. We take shortcuts under pressure.

Specialized agents don't. They follow their specification exactly, every time.

. . .

Real Implementation: What Actually Works in Production

Let me show you a concrete example: Adding a new API endpoint with full CRUD operations.

Before orchestration (my old workflow):

1. Write the Express route handler
2. Implement the controller logic
3. Add database queries
4. Create validation middleware
5. Write integration tests
6. Update API documentation
7. Add error handling
8. Review and commit

Each step requires context switching. Each step risks inconsistency. Total time:
Multiple hours spread across a day.

With orchestrated agents:

```
claude code "Add user registration endpoint with validation and tests"
```

The orchestrator delegates:

Backend agent implements the route with validation, password hashing, and JWT generation.

Security agent reviews and flags: Password hashing ✓, Email validation ✓, SQL injection protection ✓, but recommends adding rate limiting.

Testing agent creates integration tests covering successful registration, duplicate email rejection, and password validation.

Documentation agent updates the OpenAPI spec with request/response schemas.

The orchestrator presents all changes as a single, coherent pull request. I review. I approve. Done.

What changed:

- **Implementation consistency:** Perfect
- **Test coverage:** Complete
- **Documentation:** Updated automatically
- **Security review:** Built-in
- **Developer experience:** Transformed

This is the paradigm shift. I'm no longer writing repetitive code. I'm specifying what I want and reviewing what the agents produce.

Building Multi-Agent Systems That Actually Work: A 7-Step Production Guide

How to Ship Production-Ready Multi-Agent Systems Without the Technical Debt.

alirezarezvani.medium.com

```
multi-agent-system.py

import planner_agent
import coder_agent

class MultiAgentSystem:
    def __init__(self):
        self.planner = planner_agent.Planner()
        self.coder = coder_agent.CoderAgent()
        self.deployer = deployer_agent.Deployer()

    def run(self, task_description):
        plan = self.planner.plan(task_description)
        code = self.coder.write_code(plan)
        self.deployer.deploy(code)

system = MultiAgentSystem()
system.run("Build a web app")
```

. . .

The Four-Week Implementation Path

You don't need to rebuild your entire development workflow overnight. Here's how I incrementally adopted orchestrated agents:

Week 1: Start with read-only agents

Create a security auditor and a documentation analyzer. Both read-only. Run them on your codebase. Learn what they find. Refine their prompts.

```
# Your first agent
mkdir -p .claude/agents
nano .claude/agents/security-scanner.md
```

Start simple. One agent. One clear job. Read-only tools.

Week 2: Add your first write-capable agent

Pick a repetitive task you hate. For me, it was writing integration tests. Create an agent that handles it. Give it write access to test files only.

Test it on small tasks. Refine the system prompt based on what it produces.

Week 3: Build the orchestrator layer

Once you have three or four specialized agents, add orchestration. Configure how they work together. Define routing rules. Set up parallel execution.

Week 4: Add observability and optimize

Start logging agent actions. Analyze the data. Find bottlenecks. Split agents that are doing too much. Remove agents that aren't providing value.

The key: Don't try to be perfect from day one. Build incrementally. Learn what works for your codebase. Refine continuously.

. . .

Common Mistakes I Made (So You Don't Have To)

Mistake 1: Starting with too much access

My first implementation agent could run any bash command. It tried to install packages it thought it needed. One of them was malicious.

Fix: *Start with minimal permissions. Add capabilities only when you hit clear limitations.*

Mistake 2: One mega-agent for everything

I thought “one smart agent is better than multiple focused ones.” Wrong. The mega-agent started mixing concerns. It wrote Express middleware inside React components.

Fix: *One agent, one clear job. Always.*

Mistake 3: No observability from day one

I ran agents for two weeks before adding logging. I missed expensive inefficiencies and silent failures.

Fix: *Log everything immediately. Even simple JSON logs enable debugging and optimization.*

Mistake 4: Perfectionism before production use

I spent weeks refining agents on toy examples. None of it translated to real codebases with real complexity.

Fix: *Start with one real, high-value task. If it works there, scale. If not, fix it on real problems.*

. . .

Why This Changes Software Development Permanently

This isn't a temporary AI trend. It's a fundamental shift in how we build software.

We're moving from:

- Writing code → *Specifying behavior*
- Manual testing → *Automated verification*
- Documentation debt → *Continuous documentation*
- Code review → *Architecture review*

The tools are production-ready. Claude Code has native sub-agent support.

LangGraph has thirty-three percent developer adoption. AutoGen offers a complete ecosystem. CrewAI is generating millions of agents monthly.

The patterns are proven. These aren't experimental workflows. They're battle-tested approaches from distributed systems design applied to agents.

The developer experience is superior. I'm not fighting AI tools anymore. I'm coordinating a specialized workforce that handles implementation details while I focus on architecture and product decisions.

This is how software development evolves. Not by replacing developers. By **amplifying what developers can accomplish** when they have specialized, coordinated AI agents working with them.

. . .

Your Next Move: Start Small, Scale Fast

Don't bookmark this article and forget about it. Don't wait for the "*perfect time*" to experiment.

Here's what I want you to do this week:

Open your terminal. Right now.

```
# Install Claude Code
npm install -g @anthropic-ai/claude-code

# Initialize in your project
cd your-project
claude code init

# Create your first agent
mkdir -p .claude/agents
```

Create one specialized agent. Make it read-only. Make it focused. Test it on your actual codebase.

See what it finds. Refine the prompt. Get comfortable with the workflow.

Then add a second agent. Then a third. Build the orchestration layer. Add logging. Optimize based on data.

The transformation happens in practice, not in theory.

I went from seven-tab chaos to coordinated development in a month. Not because I had some special setup. Because I committed to learning the patterns and implementing them incrementally.

You can do the same. The tools exist. The patterns are proven. The only question is whether you're willing to change how you work.

10 Game-Changing CLAUDE.md Entries That Turned My Claude Code Sessions into a Coding Superpower

Tired of reading and woking through spaghetti code? The way I tackle this challenge with Claude Code Agentic Coding...

alirezarezvani.medium.com



Join the Conversation

What's blocking you from trying orchestrated agents?

Common concerns I hear:

“My codebase is too complex” → *That's exactly why you need orchestration. Complex codebases benefit most.*

“I don't have time to set this up” → *Week one takes a few hours. The time savings compound immediately.*

“Our team won't adopt this” → *Start with just yourself. Show results. Then expand.*

“I'm worried about code quality” → *Specialized agents with proper review produce more consistent code than humans.*

Drop your questions in the comments. Share what you're building. Tell me what's confusing.

I respond to every comment. This works best as a conversation.

. . .

GitHub - alirezarezvani/claude-code-tresor: A world-class collection of Claude Code utilities...

A world-class collection of Claude Code utilities: autonomous skills, expert agents, slash commands, and prompts that...

github.com

alirezarezvani/claude-code-tresor

(A collection of Claude Code utilities)
Its expert agents, slash commands, and prompts help you develop more efficiently.

Issues Stars Forks

I publish production-tested frameworks every week. No hype. Just what's working in real engineering teams.

. . .

✨ Thanks for reading! If you'd like more practical insights on AI and tech, hit **subscribe** to stay updated.

I'd also love to hear your thoughts — drop a comment with your ideas, questions, or even the kind of topics you'd enjoy seeing here next. Your input really helps shape the direction of this channel.

About the Author

Me, Alireza Rezvani work as a CTO @ an HealthTech startup in Berlin and architect AI development systems for my engineering and product teams. I write about turning individual expertise into collective infrastructure through practical automation.

Connect: [Website](#) | [LinkedIn](#)

Read more: Medium ([Reza Rezvani](#))

Explore my other open source projects: [GitHub](#)

Software Engineering

Artificial Intelligence

Programming

Developer Tools

Technology



Following ▾



Written by Reza Rezvani

1.3K followers · 77 following

As CTO of a Berlin AI MedTech startup, I tackle daily challenges in healthcare tech. With 2 decades in tech, I drive innovations in human motion analysis.

Responses (2)



Bgerby

What are your thoughts?



mohamad shahkhajeh

Nov 8



Have you measured how much context-switching with multiple AI tabs actually delays overall development time?



1 reply

[Reply](#)



Florian

Nov 5



So you wanted the latest video from IndyDevDan as well 🙌



[Reply](#)

More from Reza Rezvani



Reza Rezvani

How To Use The Reverse Prompt Engineering Magic in your favor.

The Reverse prompt engineering framework and pro tips that will help you to get the best out of your prompts in ChatGPT, Claude and Co.

★ Nov 8 🖱️ 28 💬 2



Reza Rezvani

30 Specialized Claude Code SubAgents You Need in 2026—Part 2/2: Specialized Expert-Agents

30 expert Claude Code agents: AWS (\$Ks saved), Python (95% faster), Kubernetes (99.99% uptime). Production templates for cloud native apps

★ 5d ago 🖱️ 46 💬 1





In nginity by Reza Rezvani

How Cursor and Claude Code Plugins Turned Me Into a 20x Developer – The Agentic Coding Setup That...

My Background Agent just submitted a PR that made our senior architect ask, “Who wrote this?”



Oct 14



77



Reza Rezvani

“7 Steps” How to Stop Claude Code from Building the Wrong Thing (Part 1): The Foundation of...

Learn how to stop Claude Code from rewriting your architecture with vague prompts. This guide introduces Spec-Driven Development...



Sep 17



68



2



See all from Reza Rezvani

Recommended from Medium



Dr Adrian Lucas Malec

How I Built Lightning-Fast Vector Search for Legal Documents

From 53 to 2,880 queries/sec with USearch

Oct 19



396



5



 Joe Njenga

How I Use Claude Code And Gemini CLI & CoPilot Together (Will Blow Your Mind)

Stop burning cash! Claude Code and Gemini CLI should not be competitors; if you are smart, you can see that.

★ 4d ago 🖱️ 283 💬 1



 Rick Hightower

The Claude Code Daily Handbook: Strategic AI Collaboration for Modern Developers

A Strategic Guide for Professional Developers (November 2025)

★ 6d ago 🖱️ 62



 In Medialesson by Marius Schröder

JSON vs TOON—A new era of structured input?

Why structure matters more than ever

Nov 3  797  42



 Reza Rezvani

Improving Frontend Design Through Claude Skills: Breaking Free from AI Slop

Why Every AI-Generated Interface Looks Exactly the Same



2d ago



14



1



The Atomic Architect

Architecture Patterns That Actually Scale In 2025: The Only Three You Need

Last month, a company I advised shut down.



Nov 7



1.1K



31



See more recommendations